

Python & Scripting Report

Name: Varia Neel

Enrollment No.: 250447003019

Q1 Code

```
#!/bin/bash

# Usage: ./create_users.sh users.csv

INPUT_FILE="$1"
REPORT_FILE="user_creation_report_$(date +%F_%H%M%S).log"

if [[ -z "$INPUT_FILE" ]]; then
    echo "Usage: $0 <csv_file>"
    exit 1
fi

if [[ ! -f "$INPUT_FILE" ]]; then
    echo "Error: File '$INPUT_FILE' not found!"
    exit 1
fi

# Initialize counters
created_count=0
skipped_count=0

echo "User Creation Report - $(date)" > "$REPORT_FILE"
echo "===== " >> "$REPORT_
FILE"

while IFS=, read -r username; do
    # Skip empty lines or header
    if [[ -z "$username" || "$username" == "username" ]]; then
        continue
```

```

fi

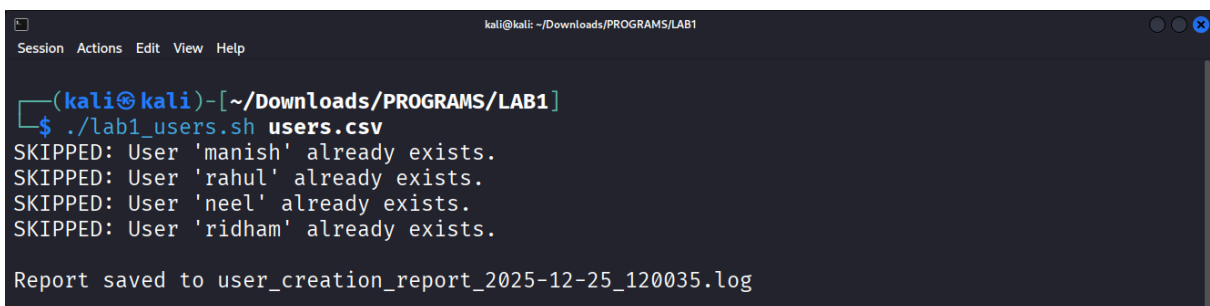
if id "$username" &>/dev/null; then
    echo "SKIPPED: User '$username' already exists." | tee -a "$REPORT_FILE"
    ((skipped_count++))
else
    if sudo useradd -m "$username"; then
        echo "CREATED: User '$username' has been created." | tee -a "$REPORT_FILE"
        ((created_count++))
    else
        echo "ERROR: Failed to create user '$username'." | tee -a "$REPORT_FILE"
    fi
fi
done < "$INPUT_FILE"

echo "===== " >> "$REPORT_FILE"
echo "Summary:" >> "$REPORT_FILE"
echo "  Users created: $created_count" >> "$REPORT_FILE"
echo "  Users skipped: $skipped_count" >> "$REPORT_FILE"

echo
echo "Report saved to $REPORT_FILE"

```

Output:



```

kali@kali: ~/Downloads/PROGRAMS/LAB1
Session Actions Edit View Help

(kali@kali)~[~/Downloads/PROGRAMS/LAB1]
$ ./lab1_users.sh users.csv
SKIPPED: User 'manish' already exists.
SKIPPED: User 'rahul' already exists.
SKIPPED: User 'neel' already exists.
SKIPPED: User 'ridham' already exists.

Report saved to user_creation_report_2025-12-25_120035.log

```

Q2 [Code](#)

```
#!/bin/bash

# The systemd unit for SSH. Use "sshd.service" for most distros.
SSH_UNIT="sshd.service"

# This file will store any suspect IPs (exceeding threshold)
OUTPUT_FILE="suspect_ips.txt"

# How many failed attempts are considered suspicious
THRESHOLD=5

# Extract failed SSH login attempts and IPs from the journal
# Use grep to extract only IPv4 addresses from the log lines.
# -o : outputs only the matching part of the line (the IP address).
# -E : enables extended regular expressions for easier pattern syntax.
# '([0-9]{1,3}\.){3}[0-9]{1,3}' : matches four groups of 1 to 3 digits separate
d by periods,
# representing IPv4 addresses (e.g., 192.168.1.10).

journalctl -u $SSH_UNIT | \
    grep "Failed password" | \                                # Only lines about failed SSH
passwords
    grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' > all_failed_ips.txt # Extract just the I
P addresses

# Count how many times each IP appears and sort by count (descending)
sort all_failed_ips.txt | uniq -c | sort -nr > temp_ips.txt

# Write only those IPs with count >= threshold to the output file
awk -v threshold="$THRESHOLD" '{if ($1 >= threshold) print $2}' temp_ip
s.txt > "$OUTPUT_FILE"

# If any suspect IPs found, send an email alert (customize as needed)
# Check if the file specified by $OUTPUT_FILE exists and is not empty (-s c
hecks size > 0)
if [ -s "$OUTPUT_FILE" ]; then
    # If the file has content, send an email alert with subject "Suspect SSH L
ogin Attempts Detected"
```

```

# The email body consists of the contents of $OUTPUT_FILE (list of suspect IPs)
# Replace "admin@example.com" with the actual alert recipient's email address
# Ensure mail has been setup
mail -s "Suspect SSH Login Attempts Detected" admin@example.com < "$OUTPUT_FILE"
fi
# Remove temporary files or you can keep them if need be
rm all_failed_ips.txt temp_ips.txt

```

Output:

```

(kali@kali)-[~/Downloads/PROGRAMS/LAB2]
$ ./ssh_log_alert.sh

```

Q3 Code

```

#!/bin/bash

# Regex File Search, Archive, and Backup Script

# Usage: ./search_archive_backup.sh <search_dir> <regex_pattern>
# Example: ./search_archive_backup.sh /home/user ".*\.(jpg|png|jpeg)$"
# If using above example ensure you have pictures with .png or .jpg extensions present in the folder
# You can modify the above regex to work with file extensions of your choice note that each extension is separated by a |
# Also logs and archive will be created in current directory where the shell script is run from.

# Enable strict mode: exit on error, treat unset variables as errors, and propagate pipe errors
set -euo pipefail

# -----
# Assign input parameters to variables

```

```

# "$1" is the first argument (search directory)
# "$2" is the second argument (regex pattern)
# -----
SEARCH_DIR="$1" # Directory in which to search for files
REGEX_PATTERN="$2" # Regular expression pattern to match files (POSIX-
extended)

# Generate a unique timestamp based on date and time (YYYYMMDD_HHM
MSS format)
TIMESTAMP="$(date '+%Y%m%d_%H%M%S')"
```

Set archive and log filenames using the timestamp, ensuring unique outp
ut files

```

ARCHIVE_NAME="backup_${TIMESTAMP}.tar.gz" # Final archive
filename
LOGFILE="backup_${TIMESTAMP}.log" # Logfile to record s
cript actions
TMP_FILE_LIST="matched_files_${TIMESTAMP}.txt" # Temporary fil
e to store the matched filenames
```

Define a logging function

Prepends a timestamp to messages and appends them to the logfile

Also prints to the terminal for user feedback

```

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOGFILE"
}

# -----
# Check if the target directory exists
# If not, log an error and exit the script (exit code 1)
# -----
if [[ ! -d "$SEARCH_DIR" ]]; then
    log "Error: Directory '$SEARCH_DIR' not found. Exiting."
    exit 1
fi
```

```

# Log the beginning of the search action, along with the parameters used
log "Starting file search in: $SEARCH_DIR"
log "Using regex pattern: $REGEX_PATTERN"

# -----
# Perform the regex file search using 'find'
# - Searches recursively (-type f for files)
# - Uses POSIX-extended regular expression matching (-regextype posix-extended)
# - Writes all matching file paths to the temporary file list
# If the find command fails, log an error and exit (exit code 2)
# -----
if ! find "$SEARCH_DIR" -type f -regextype posix-extended -regex "$REGEX_PATTERN" > "$TMP_FILE_LIST"; then
    log "Error: Error occurred while searching for files."
    exit 2
fi

# -----
# Check if the search produced any results
# -s tests if the file is non-empty
# If no matching files were found, clean up temporary files and exit gracefully
# -----
if [[ ! -s "$TMP_FILE_LIST" ]]; then
    log "No files matched the pattern. Nothing to archive."
    rm -f "$TMP_FILE_LIST"
    exit 0
fi

# Log how many files matched (using 'wc -l' to count lines in the file list)
log "Found $(wc -l < "$TMP_FILE_LIST") files. Proceeding to archive."

# -----
# Archive and compress the matched files using tar and gzip (-czf options)

```

```

# see which command works otherwise you may need to install. mostly bot
h are present by default
# -c: create a new archive
# -z: filter the archive through gzip for compression
# -f: specify the output filename (-T: read file names from the temporary lis
t)
# Output errors to the logfile, check success/failure for logging
# -----
-----

if tar -czf "$ARCHIVE_NAME" -T "$TMP_FILE_LIST" 2>>"$LOGFILE"; then
    log "Archive created: $ARCHIVE_NAME"
else
    log "Error: Failed to create archive."
    rm -f "$TMP_FILE_LIST"
    exit 3
fi

# -----
-----

# Remove the temporary file list (housekeeping)
# Ensure temporary files do not accumulate
# -----
-----

rm -f "$TMP_FILE_LIST"
log "Temporary file list removed."
log "Backup and archiving complete!"

# End the script with success status (0)
exit 0

```

Output:

```

(kali@kali)-[~/Downloads/PROGRAMS/LAB3]
$ ./search_archive_backup.sh /home/kali/Pictures '*.*(jpg|png|jpeg)$'
[2025-12-25 12:12:40] Starting file search in: /home/kali/Pictures
[2025-12-25 12:12:40] Using regex pattern: *.*(jpg|png|jpeg)$
[2025-12-25 12:12:40] No files matched the pattern. Nothing to archive.

```

Q4 Code

```
#!/bin/bash
# Usage: Run this script on a Linux system to generate a network informati
on report.
# The script collects network interface details, checks connectivity to speci
fied servers,
# allows user input for additional diagnostics, and formats all data into a pri
ntable report.
#
# Disclaimer: This script uses the 'ifconfig' command. On Ubuntu or Debian
-based systems,
# if 'ifconfig' is not installed, please install it using:
# sudo apt install net-tools
#
# Note: Printing the report is optional and requires the 'lp' command to be c
onfigured.

# Define the file name that will store the generated network report
REPORT="network_report.txt"

# Write the report heading into the file (overwrite if exists)
echo "Network Information Report" > $REPORT
echo "=====" >> $REPORT
echo "" >> $REPORT

# Extract list of network interface names from ifconfig output
# The command first filters lines starting with a device name (in ifconfig out
put)
# and then awk selects the first token which is the interface name
interfaces=$(ifconfig | grep '^[a-zA-Z0-9]' | awk '{print $1}')

# Append a section title about network interfaces and their IPs
echo "Network Interfaces and IP Information:" >> $REPORT

# Iterate through each interface found
for iface in $interfaces; do
    # Append the interface name to the report
    echo "Interface: $iface" >> $REPORT

```



```

# Extract the IPv4 address for the interface
# Grep 'inet ' lines and awk prints the second field (IP address)
ifconfig $iface | grep 'inet ' | awk '{print " IP Address: "$2}' >> $REPORT

# Extract the netmask by finding 'netmask' in ifconfig output and printing the fourth field
ifconfig $iface | grep 'netmask' | awk '{print " Netmask: "$4}' >> $REPORT

# Extract the MAC (hardware) address by looking for 'ether' line and printing second field
ifconfig $iface | grep 'ether' | awk '{print " MAC Address: "$2}' >> $REPORT

# Add a blank line to visually separate interface sections in the report
echo "" >> $REPORT
done

# Define an array of key servers/IPs to test connectivity
servers=("8.8.8.8" "1.1.1.1" "www.google.com")

# Append connectivity check section header to report
echo "Connectivity Check:" >> $REPORT

# Loop over all servers in the array
for server in "${servers[@]}; do
    # Ping the server once, with timeout of 1 second.
    # Redirect all output to null, we only care about success or failure
    if ping -c 1 -W 1 $server &>/dev/null; then
        # If ping success, log that server is reachable
        echo "$server is reachable." >> $REPORT
    else
        # Otherwise, log that server is unreachable
        echo "$server is unreachable." >> $REPORT
    fi
done

# Add a blank line before user diagnostics

```

```

echo "" >> $REPORT

# Prompt the user to enter a hostname or IP address for additional diagnostics
read -p "Enter hostname or IP to ping for diagnostics: " target

# Ping the user provided target once and check success
if ping -c 1 -W 1 $target &>/dev/null; then
    # If reachable, log success
    echo "$target is reachable." >> $REPORT
else
    # If not reachable, log failure
    echo "$target is unreachable." >> $REPORT
fi

# Add a blank line after user diagnostics section
echo "" >> $REPORT

# Inform the user that report generation is complete and saved
echo "Report generated and saved to $REPORT"

# Optionally, print the report using 'lp' command (commented out)
# lp $REPORT

```

Output

```

(kali@kali)~[~/Downloads/PROGRAMS/LAB4]
$ ./network_report.sh
Enter hostname or IP to ping for diagnostics: 192.168.74.174
Report generated and saved to network_report.txt

```

Q5 Code

```

# This program collects user records, stores them in a list and dictionary,
# and provides functions for displaying, filtering, and searching the data.

def add_record(records, record_dict):
    # Prompt the user for required fields
    name = input("Enter name: ")

```

```

age = int(input("Enter age: "))
city = input("Enter city: ")

# Add the new record to the list (for iteration/ordering)
records.append({'name': name, 'age': age, 'city': city})

# Add the record to the dictionary (for fast lookup by name)
record_dict[name] = {'age': age, 'city': city}
print("Record added.\n")

def display_records(records):
    # Display all records in a readable format
    print("\nAll Records:")
    for rec in records: # Iterate through the list using a for loop
        print(f"Name: {rec['name']}, Age: {rec['age']}, City: {rec['city']}")
    print()

def filter_by_age(records, min_age):
    # Show only those records where age meets the minimum threshold
    print(f"\nRecords with age >= {min_age}:")
    for rec in records: # Iterate through all records
        if rec['age'] >= min_age: # Use 'if' for conditional filtering
            print(f"Name: {rec['name']}, Age: {rec['age']}, City: {rec['city']}")
    print()

def search_by_name(record_dict):
    # Allow user to search for a record by name (dictionary lookup)
    name = input("Enter name to search: ")
    if name in record_dict: # Use 'if-else' for checking existence
        print(f"Found: Name: {name}, Age: {record_dict[name]['age']}, City: {r
ecord_dict[name]['city']}\n")
    else:
        print("Name not found.\n")

def main():
    records = [] # List to store all record dicts (ordered)
    record_dict = {} # Dictionary for fast name-based lookups

```

```

while True:      # Menu loop runs until 'Exit' is selected
    print("Menu:")
    print("1. Add Record")
    print("2. Display All Records")
    print("3. Filter Records by Age")
    print("4. Search by Name")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    # Use 'if-elif-else' to select menu option
    if choice == "1":
        add_record(records, record_dict)
    elif choice == "2":
        display_records(records)
    elif choice == "3":
        min_age = int(input("Enter minimum age: "))
        filter_by_age(records, min_age)
    elif choice == "4":
        search_by_name(record_dict)
    elif choice == "5":
        print("Exiting program.")
        break # Exit the while loop and program
    else:
        print("Invalid choice, try again.\n")

# Python standard entry point check to allow this script to run directly
if __name__ == "__main__":
    main()

```

Output

```

PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS> python -u "d:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB5\python_basics.py"
Menu:
1. Add Record
2. Display All Records
3. Filter Records by Age
4. Search by Name
5. Exit
Enter your choice (1-5): 1
Enter name: Neel
Enter age: 22
Enter city: Goa
Record added.

Menu:
1. Add Record
2. Display All Records
3. Filter Records by Age
4. Search by Name
5. Exit
Enter your choice (1-5): 2

All Records:
Name: Neel, Age: 22, City: Goa

Menu:
1. Add Record
2. Display All Records
3. Filter Records by Age
4. Search by Name
5. Exit
Enter your choice (1-5): █

```

Q6 Code

1.TCP

```

import socket

class TCPServer:
    def __init__(self, host='127.0.0.1', port=9999):
        # Initialize the server with host and port
        self.host = host
        self.port = port
        # Create a TCP socket using IPv4 addressing
        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_
STREAM)
        # Bind the socket to the given host and port
        self.server_socket.bind((self.host, self.port))
        # Start listening for incoming connection requests (1 client at a tim
e)
        self.server_socket.listen(1)
        print(f"TCP Server listening on {self.host}:{self.port}")

```

```

def run(self):
    # Start the main loop to accept and handle client connections
    while True:
        # Accept a new client connection
        client_socket, addr = self.server_socket.accept()
        print(f"Connected by {addr}")
        # Receive up to 1024 bytes from the client
        data = client_socket.recv(1024).decode()
        print("Received:", data)
        # Send back the data in uppercase as a response
        client_socket.sendall(data.upper().encode())
        # Close the client socket after handling the request
        client_socket.close()

if __name__ == "__main__":
    # Create the server object and start it
    server = TCPServer()
    server.run()

#Client
import socket

class TCPClient:
    def __init__(self, server_host='127.0.0.1', server_port=9999):
        # Initialize with server's host and port details
        self.server_host = server_host
        self.server_port = server_port

    def send(self, message):
        # Create a TCP socket for connecting to server
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            # Connect to the server
            sock.connect((self.server_host, self.server_port))
            # Send the message to the server
            sock.sendall(message.encode())
            # Receive the response from the server
            response = sock.recv(1024).decode()

```

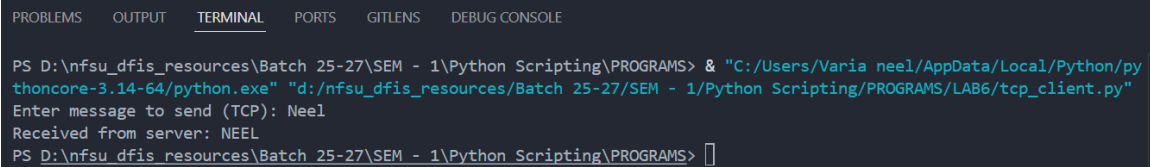
```

        print("Received from server:", response)

if __name__ == "__main__":
    # Create client object and send user input to server
    client = TCPClient()
    msg = input("Enter message to send (TCP): ")
    client.send(msg)

```

Output



```

PROBLEMS  OUTPUT  TERMINAL  PORTS  GITLENS  DEBUG CONSOLE
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS> & "C:/Users/Varia neel/AppData/Local/Python/py
thoncore-3.14-64/python.exe" "d:/nfsu_dfis_resources/Batch 25-27/SEM - 1/Python Scripting/PROGRAMS/LAB6/tcp_client.py"
Enter message to send (TCP): Neel
Received from server: NEEL
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS>

```

2.UDP

```

import socket

class UDPClient:
    def __init__(self, server_host='127.0.0.1', server_port=9999):
        # Initialize with server's host and port configuration
        self.server_host = server_host
        self.server_port = server_port

    def send(self, message):
        # Create a UDP socket for sending/receiving datagrams
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as soc
        k:
            # Send the encoded message to the server's address
            sock.sendto(message.encode(), (self.server_host, self.server_po
            rt))

            # Wait for and receive the response from server
            response, _ = sock.recvfrom(1024)
            print("Received from server:", response.decode())

if __name__ == "__main__":
    # Create the client and send user input as a message
    client = UDPClient()

```

```

msg = input("Enter message to send (UDP): ")
client.send(msg)

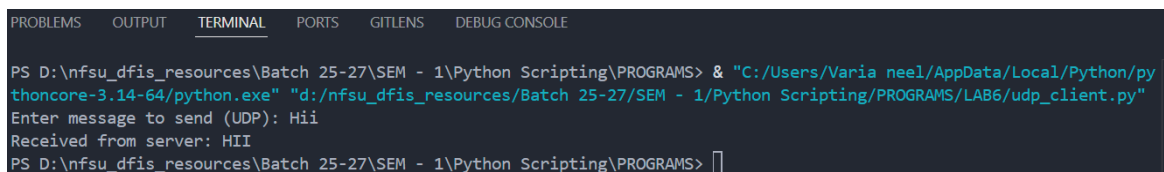
#Server
import socket

class UDPServer:
    def __init__(self, host='127.0.0.1', port=9999):
        # Initialize the server with specified host and port
        self.host = host
        self.port = port
        # Create a UDP socket using IPv4
        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_
DGRAM)
        # Bind the socket to host and port for receiving data
        self.server_socket.bind((self.host, self.port))
        print(f"UDP Server listening on {self.host}:{self.port}")

    def run(self):
        # Server loop: continuously listen for client datagrams
        while True:
            # Receive data and the address it came from (client)
            data, addr = self.server_socket.recvfrom(1024)
            print(f"Received from {addr}: {data.decode()}")
            # Send back the received data in uppercase to client
            self.server_socket.sendto(data.upper(), addr)

if __name__ == "__main__":
    # Create and run the UDP server
    server = UDPServer()
    server.run()

```



```

PROBLEMS OUTPUT TERMINAL PORTS GITLENS DEBUG CONSOLE
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS> & "C:/Users/Varia neel/AppData/Local/Python/py
thoncore-3.14-64/python.exe" "d:/nfsu_dfis_resources/Batch 25-27/SEM - 1/Python Scripting/PROGRAMS/LAB6/udp_client.py"
Enter message to send (UDP): Hii
Received from server: HII
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS> 

```



```
PROBLEMS  OUTPUT  TERMINAL  PORTS  GITLENS  DEBUG CONSOLE

PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS> & "C:/Users/Varia neel/AppData/Local/Python/py
thoncore-3.14-64/python.exe" "d:/nfsu_dfis_resources/Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB6\udp_client.py"
Enter message to send (UDP): Hii
Received from server: HII
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS> []
```

Q7. Code

```
import os                # Import the os module for interacting with the op
erating system
import logging           # Import logging to record messages to a log file
from PIL import Image    # Import the Image class from PIL (Pillow) fo
r image processing
import numpy as np       # Import numpy as np for numerical operatio
ns on image arrays

# Setup logging to a file named 'analysis_log.txt', with INFO level and a stan
dard message format
logging.basicConfig(
    filename='analysis_log.txt', # Log file name
    level=logging.INFO,         # Log level set to INFO
    format='%(asctime)s %(levelname)s: %(message)s' # Log message for
mat: timestamp, level, message
)

def read_text_file(filepath):
    """
    Safely reads a text file, handling exceptions.
    """
    try:
        # Try to open the file at the given path in read mode
        with open(filepath, 'r') as f:
            data = f.read()          # Read all content from the file into a string
            logging.info(f"Successfully read text file: {filepath}") # Log successful
read
    return data                    # Return the string data
except FileNotFoundError:
    logging.error(f"File not found: {filepath}") # Log if the file does not ex
```

```

ist
    except Exception as e:
        logging.error(f"Error reading {filepath}: {str(e)}") # Log any other exc
        eptions
    return None # Return None if reading failed

def analyze_image(filepath):
    """
    Extract pixels and compute statistics from an image using PIL.
    """
    try:
        # Try to open the image file at the given path
        with Image.open(filepath) as img:
            img_array = np.array(img) # Convert the image to a numpy arra
            y for analysis
            max_pixel = np.max(img_array) # Find the maximum pixel value in
            the array
            mean_pixel = np.mean(img_array) # Compute the mean (average) o
            f all pixel values
            logging.info(
                f"Image: {filepath} | Max Pixel Value: {max_pixel} | Mean Pixel Val
                ue: {mean_pixel:.2f}"
            ) # Log the computed statistics
            return {
                'max_pixel': max_pixel, # Return max pixel value in a dictionary
                'mean_pixel': mean_pixel, # Return mean pixel value in the same
                dictionary
            }
    except FileNotFoundError:
        logging.error(f"Image not found: {filepath}") # Log if the image file is
        missing
    except Exception as e:
        logging.error(f"Error processing image {filepath}: {str(e)}") # Log any
        image processing errors
    return None # Return None if processing failed

def main():
    # Example file paths for text and image

```

```

text_path = 'example.txt'          # Path to the text file to read
image_path = 'example.jpg'        # Path to the image file to analyze

# Read text data from disk
text_data = read_text_file(text_path) # Call function to read text file

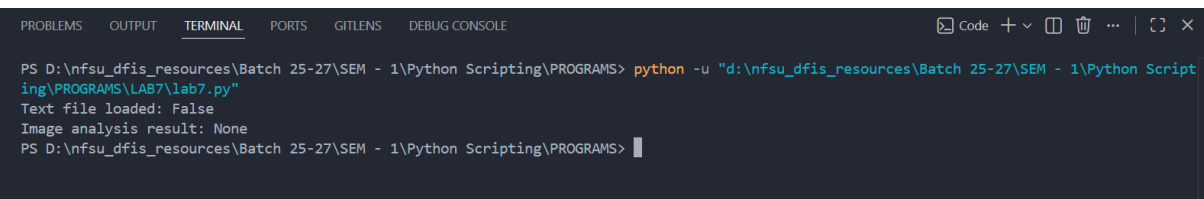
# Analyze the image file for pixel statistics
img_stats = analyze_image(image_path) # Call function to analyze the i
mage

# Optional: Print whether text file loaded successfully
print(f"Text file loaded: {text_data is not None}") # Print status of text fil
e loading
# Optional: Print the analysis result of the image
print(f"Image analysis result: {img_stats}")        # Print image statistics
(dict or None)

# Check if this script is being run directly
if __name__ == "__main__":
    main()          # Call the main function if script is executed d
irectly

```

Output



```

PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS> python -u "d:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB7\lab7.py"
Text file loaded: False
Image analysis result: None
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS>

```

Q8. [Code](#)

Q9. [Code](#)

```

# diag.ps1
# PowerShell script for local diagnostics (cross-platform)
# Demonstrates advanced functions, param block, output processing, and

```

ByRef argument usage

```
function Invoke-LocalDiagnostics {
    [CmdletBinding()]
    param (
        # ByRef argument allows function to update caller's hashtable
        [ref]$SummaryHash
    )

    # Cross-platform computer name
    $computerName = [System.Net.Dns]::GetHostName()

    # Get basic OS details for macOS/Linux
    try {
        # Get-ComputerInfo not supported on macOS
        $osName    = (uname -s)
        $osVersion = (sw_vers -productVersion)
        $architecture = (uname -m)
        $osInfo = [PSCustomObject]@{
            OsName      = $osName
            OsArchitecture = $architecture
            OsVersion    = $osVersion
        }
    } catch {
        # Fallback values if commands fail
        $osInfo = [PSCustomObject]@{
            OsName      = "Unknown"
            OsArchitecture = "Unknown"
            OsVersion    = "Unknown"
        }
    }

    # Get top 5 CPU-consuming processes on the system
    $processes = Get-Process | Sort-Object CPU -Descending | Select-Object
    -First 5 Name, CPU, Id

    # Create a custom object to hold diagnostic results
    $result = [PSCustomObject]@{
```

```

    Computer    = $computerName          # Machine name (cross-
platform)
    OS          = $osInfo.OsName          # OS Name
    Architecture = $osInfo.OsArchitecture # OS Architecture
    Version      = $osInfo.OsVersion      # OS Version
    TopProcesses = $processes             # Top processes collectio
n
}

# Prepare or update the summary hash table, using -ByRef for direct upd
ate
$summary = $SummaryHash.Value
$summary[$computerName] = @{
    OS      = $result.OS
    LastBoot = "N/A"          # macOS: last boot not handled h
ere
    ProcCount = $result.TopProcesses.Count
}
$SummaryHash.Value = $summary

# Return diagnostics object for inspection or output
return $result
}

# Instructions:
# 1. In pwsh, load this script: . ./diag.ps1
# 2. Prepare an empty summary hash table: $summary = @{}
# 3. Call the function: $diagnostic = Invoke-LocalDiagnostics -SummaryHa
sh ([ref]$summary)
# 4. View results:
#   $diagnostic | Format-List
#   $summary | Format-Table

```

Output

```

Administrator: Windows PowerShell
Directory: D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB9

Mode                LastWriteTime         Length Name
----                -
-a-----          20-11-2025      19:05           2405 diag.ps1

PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB9> .\diag.ps1
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB9> $summary = @{}
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB9> $diagnostic = Invoke-LocalDiagnostics -SummaryHash ([ref]$summary)
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB9> $diagnostic | Format-List

Computer       : LAPTOP-FMI0HU75
OS              : Unknown
Architecture   : Unknown
Version        : Unknown
TopProcesses   : @{Name=WmiPrvSE; CPU=8438.46875; Id=23832}, @({Name=System; CPU=6237.921875; Id=4},
                  @({Name=LightingService; CPU=5785.109375; Id=5900}, @({Name=svchost; CPU=4603.359375; Id=5344})...)

PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB9> $summary | Format-Table

Name                Value
----                -
LAPTOP-FMI0HU75      {ProcCount, OS, LastBoot}

```

Q10. Code

```

# csv.ps1
# PowerShell script that automates reading from and writing to CSV files
# Demonstrates param blocks, error handling, file I/O, and structured processing for cross-platform use

param(
    [string]$InputCsv = "./users.csv",          # Path to input CSV file
    [string]$OutputCsv = "./processed-users.csv", # Path to output CSV file
    [string]$ErrorLog = "./errors.log"          # Path to error log file
)

# Remove output and error log files from previous runs, silencing errors if files do not exist
Remove-Item $OutputCsv, $ErrorLog -ErrorAction SilentlyContinue

try {
    # Check if the input file exists. If not, throw an error.
    if (-not (Test-Path $InputCsv)) {
        throw "Input CSV file '$InputCsv' not found."
    }
}

```

```

# Import user data from CSV into array of objects
$users = Import-Csv -Path $InputCsv
$results = @() # Initialize empty array to store result objects

# Process each user record from the CSV
foreach ($user in $users) {
    try {
        # Example transformation: build full name
        $fullName = "$($user.FirstName) $($user.LastName)"

        # Add processed info to results array
        $results += [PSCustomObject]@{
            Username = $user.Username
            FullName = $fullName
            Status = "Processed"
        }
    }
    catch {
        # On error, record status and write error details to log
        $results += [PSCustomObject]@{
            Username = $user.Username
            FullName = ""
            Status = "Error: $($_.Exception.Message)"
        }
        # Append error details to the error log file with date/time
        "`n[$(Get-Date)] Error processing '$($user.Username)': $($_.Exception.Message)" | Out-File -FilePath $ErrorLog -Append
    }
}

# Export processed results to new CSV file
$results | Export-Csv -Path $OutputCsv -NoTypeInfo

# Check if export was successful, throw exception if not
if (!$?) {
    throw "Failed to write processed data to $OutputCsv"
}
}

```

```

catch {
    # Top-level error capture, logs fatal script errors to error log file
    "[$(Get-Date)] Fatal script error: $($_.Exception.Message)" | Out-File -File
    ePath $ErrorLog -Append
}

# Show completion message with output and error log file info
Write-Host "Processing complete. Output: $OutputCsv, Errors (if any): $ErrorLog"

```

Output

The screenshot shows a Windows PowerShell terminal window titled "Administrator: Windows PowerShell". The prompt is at the directory `D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB10`. The user has run a script `csv.ps1`, which has completed. The output shows that the script processed users and generated two CSV files: `processed-users.csv` (98 bytes) and `users.csv` (59 bytes). The terminal also shows the command `dir` being used to list the directory contents.

```

PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB10> .\csv.ps1
Processing complete. Output: ./processed-users.csv, Errors (if any): ./errors.log
PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB10> dir

Directory: D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB10

Mode                LastWriteTime         Length Name
----                -
-a----          20-11-2025   19:05           2468 csv.ps1
-a----          26-12-2025   00:07             98 processed-users.csv
-a----          20-11-2025   19:05             59 users.csv

PS D:\nfsu_dfis_resources\Batch 25-27\SEM - 1\Python Scripting\PROGRAMS\LAB10>

```